

OS_LAB

SKILL3

1. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

history.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_HISTORY 100
#define BUFFER_SIZE 1024

char *history[MAX_HISTORY];
int history_count = 0;
int history_index = 0;

/* Read one character without waiting for Enter */
char getch(void)
{
    struct termios oldattr, newattr;
    char ch;

    tcgetattr(STDIN_FILENO, &oldattr);

    newattr = oldattr;
    newattr.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newattr);

    ch = getchar();

    tcsetattr(STDIN_FILENO, TCSANOW, &oldattr);

    return ch;
}
```

```

/* Add command to history */
void add_history(const char *command)
{
    if (strlen(command) == 0)
        return;
    if (history_count < MAX_HISTORY)
    {
        history[history_count] = malloc(strlen(command) + 1);

        if (history[history_count] == NULL)
        {
            perror("malloc");
            exit(EXIT_FAILURE);
        }

        strcpy(history[history_count], command);
        history_count++;
    }
}

```

```

/* Display history */
void show_history(void)
{
    printf("\n\nCommand History:\n");

    for (int i = 0; i < history_count; i++)
    {
        printf("%d: %s\n", i + 1, history[i]);
    }
}

```

```

/* Free allocated memory */
void free_history(void)
{
    for (int i = 0; i < history_count; i++)
    {
        free(history[i]);
    }
}

```

```

int main(void)
{
    char buffer[BUFFER_SIZE];

```

```

int position;

printf("Simple Command History\n");
printf("Type commands. Use UP/DOWN arrows for history.\n");
printf("Type 'history' to display history.\n");
printf("Type 'exit' to quit.\n\n");

while (1)
{
    printf("myshell> ");
    fflush(stdout);
    position = 0;
    buffer[0] = '\0';
    history_index = history_count;

    while (1)
    {
        char ch = getch();

        /* ENTER */
        if (ch == '\n' || ch == '\r')
        {
            buffer[position] = '\0';
            printf("\n");
            break;
        }

        /* BACKSPACE */
        else if (ch == 127 || ch == 8)
        {
            if (position > 0)
            {
                position--;
                buffer[position] = '\0';

                printf("\b \b");
                fflush(stdout);
            }
        }

        /* ESCAPE SEQUENCE */
        else if (ch == 27)
        {

```

```

char ch1 = getch();

if (ch1 == '[')
{
    char ch2 = getch();

    /* UP ARROW */
    if (ch2 == 'A')
    {
        if (history_index > 0)
        {
            history_index--;

            strcpy(buffer, history[history_index]);
            position = strlen(buffer);

            printf("\r\033[Kmyshell> %s", buffer);
            fflush(stdout);
        }
    }

    /* DOWN ARROW */
    else if (ch2 == 'B')
    {
        if (history_index < history_count - 1)
        {
            history_index++;

            strcpy(buffer, history[history_index]);
            position = strlen(buffer);

            printf("\r\033[Kmyshell> %s", buffer);
            fflush(stdout);
        }
        else
        {
            history_index = history_count;
            buffer[0] = '\0';
            position = 0;

            printf("\r\033[Kmyshell> ");
            fflush(stdout);
        }
    }
}

```

```

    }
}

/* NORMAL CHARACTER */
else if (ch >= 32 && ch <= 126)
{
    if (position < BUFFER_SIZE - 1)
    {
        buffer[position++] = ch;
        buffer[position] = '\0';

        putchar(ch);
        fflush(stdout);
    }
    else
    {
        printf("\nBuffer full!\n");
    }
}

}

if (strcmp(buffer, "exit") == 0)
{
    break;
}

if (strcmp(buffer, "history") == 0)
{
    show_history();
}
else if (strlen(buffer) > 0)
{
    add_history(buffer);
}
}

free_history();

printf("Program terminated.\n");

return 0;
}

```

memory.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

/* Node structure for linked list */
struct Node
{
    int data;
    struct Node *next;
};

/* Insert node at beginning */
void insert(struct Node **head, int value)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(EXIT_FAILURE);
    }

    newNode->data = value;
    newNode->next = *head;

    *head = newNode;
}

/* Display linked list */
void display(struct Node *head)
{
    struct Node *temp = head;

    printf("Linked List: ");

    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
}
```

```

    }

    printf("NULL\n");
}

/* Free linked list */
void free_list(struct Node *head)
{
    struct Node *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;

        free(temp);
    }
}

int main(void)
{
    int *buffer;
    int size = INITIAL_SIZE;

    printf("Dynamic Memory Management\n\n");

    /* Allocate memory */
    buffer = malloc(size * sizeof(int));

    if (buffer == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Allocated memory for %d integers.\n", size);

    /* Store values */
    for (int i = 0; i < size; i++)
    {
        buffer[i] = i + 1;
    }

    printf("Values stored successfully.\n");
}

```

```

/* Resize array */
size = 20;

int *temp = realloc(buffer, size * sizeof(int));

if (temp == NULL)
{
    printf("Memory reallocation failed!\n");
    free(buffer);
    return 1;
}

buffer = temp;

printf("Array resized to %d integers.\n", size);

/* Add more values */
for (int i = 10; i < size; i++)
{
    buffer[i] = i + 1;
}

printf("Array values:\n");

for (int i = 0; i < size; i++)
{
    printf("%d ", buffer[i]);
}

printf("\n\n");

/* Linked list */
struct Node *head = NULL;

insert(&head, 10);
insert(&head, 20);
insert(&head, 30);
insert(&head, 40);

display(head);

/* Release linked list memory */
free_list(head);

```



```

/* Release dynamic array */
free(buffer);

printf("\nAll dynamically allocated memory released.\n");

return 0;
}

```

```

lahari@DESKTOP-AIC33F7:~$ cd Skill
lahari@DESKTOP-AIC33F7:~/Skill$ mkdir skill3
lahari@DESKTOP-AIC33F7:~/Skill$ cd skill3
lahari@DESKTOP-AIC33F7:~/Skill/skill3$ mkdir command_history
lahari@DESKTOP-AIC33F7:~/Skill/skill3$ cd command_history
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ pwd
/home/lahari/Skill/skill3/command_history
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ nano history.c
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ gcc history.c -o history
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ ls
history  history.c
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ ./history
Simple Command History
Type commands. Use UP/DOWN arrows for history.
Type 'history' to display history.
Type 'exit' to quit.

```

```

myshell> Quit
myshell> history

```

```

Command History:
1: Quit
myshell> Quit                ./history
myshell> one
myshell> two
myshell> three
myshell> one
myshell> exit
Program terminated.

```

```

lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ nano memory.c
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ gcc memory.c -o memory
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ ./memory
Dynamic Memory Management

```

```

Allocated memory for 10 integers.
Values stored successfully.
Array resized to 20 integers.
Array values:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Linked List: 40 -> 30 -> 20 -> 10 -> NULL

All dynamically allocated memory released.

```

```

lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ sudo apt update
[sudo] password for lahari:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [973 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1230 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [207 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.3 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1203 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [287 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [180 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1689 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [241 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [338 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5760 B]
Get:19 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.7 kB]
Fetched 7256 kB in 5s (1428 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
37 packages can be upgraded. Run 'apt list --upgradable' to see them.

```

```

lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ gcc -g memory.c -o memory
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$ valgrind --leak-check=full ./memory
==1628== Memcheck, a memory error detector
==1628== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1628== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1628== Command: ./memory
==1628==
Dynamic Memory Management

Allocated memory for 10 integers.
Values stored successfully.
Array resized to 20 integers.
Array values:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Linked List: 40 -> 30 -> 20 -> 10 -> NULL

All dynamically allocated memory released.
==1628==
==1628== HEAP SUMMARY:
==1628==    in use at exit: 0 bytes in 0 blocks
==1628==   total heap usage: 7 allocs, 7 frees, 1,208 bytes allocated
==1628==
==1628== All heap blocks were freed -- no leaks are possible
==1628==
==1628== For lists of detected and suppressed errors, rerun with: -s
==1628== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
lahari@DESKTOP-AIC33F7:~/Skill/skill3/command_history$

```